

Shopify Integration Guide — Luciteria Collector Cabinet

Overview

The Luciteria Collector Cabinet is a Shopify Remix app that manages element collection subscriptions. This guide documents every integration point a developer needs to connect the prototype to the live Shopify store.

Current State: Prototype uses mock data + real SKU mapping CSV.

Target State: Full Shopify API integration with PostgreSQL database.

Quick Start for Developers

1. Copy `.env.example` to `.env`
 2. Set `APP_MODE=production`
 3. Fill in all `SHOPIFY_*` credentials
 4. Switch Prisma to PostgreSQL (see Database section)
 5. Run `npx prisma migrate deploy`
 6. Follow the integration checklist in `DEVELOPER_INTEGRATION_CHECKLIST.md`
-

Architecture Overview



Required OAuth Scopes

```
read_products, write_products
read_customers, write_customers
read_orders, write_orders
read_draft_orders, write_draft_orders
read_inventory, write_inventory
read_own_subscription_contracts, write_own_subscription_contracts
```

API Endpoints Needed

Products

Operation	Endpoint	Method
List products	/admin/api/2024-10/products.json	GET
Get product	/admin/api/2024-10/products/{id}.json	GET
Get variants	/admin/api/2024-10/products/{id}/variants.json	GET
Get inventory	/admin/api/2024-10/inventory_levels.json	GET
Adjust inventory	/admin/api/2024-10/inventory_levels/adjust.json	POST

Customers

Operation	Endpoint	Method
List customers	/admin/api/2024-10/customers.json	GET
Search customers	/admin/api/2024-10/customers/search.json	GET
Get customer	/admin/api/2024-10/customers/{id}.json	GET
Get metafields	/admin/api/2024-10/customers/{id}/metafields.json	GET
Set metafield	/admin/api/2024-10/customers/{id}/metafields.json	POST

Orders

Operation	Endpoint	Method
Create draft order	<code>/admin/api/2024-10/draft_orders.json</code>	POST
Complete draft	<code>/admin/api/2024-10/draft_orders/{id}/complete.json</code>	PUT
Add fulfillment	<code>/admin/api/2024-10/orders/{id}/fulfillments.json</code>	POST
List orders	<code>/admin/api/2024-10/orders.json</code>	GET

Subscriptions (GraphQL only)

Operation	Mutation/Query
Create contract	<code>subscriptionDraftCreate</code> → <code>subscriptionDraftCommit</code>
Update contract	<code>subscriptionContractUpdate</code>
Get contract	<code>subscriptionContract(id:)</code>
Billing attempt	<code>subscriptionContractAttemptBilling</code>

Webhook Configuration

Required Webhook Topics

Topic	Handler	Purpose
<code>products/update</code>	Update local product data	Price, status, tag changes
<code>products/delete</code>	Archive product locally	Remove from subscription pool
<code>inventory_levels/update</code>	Update stock levels	Trigger OOS shifting if needed
<code>customers/create</code>	Create local customer	Onboarding flow trigger
<code>customers/update</code>	Sync customer data	Email, name changes
<code>customers/delete</code>	Archive customer	Cancel subscription, cleanup
<code>orders/create</code>	Track subscription order	Link order to shipment
<code>orders/paid</code>	Confirm payment	Trigger fulfillment
<code>orders/fulfilled</code>	Update tracking	Notify customer, update collection
<code>subscription_contracts/create</code>	New subscription	Run initial assignment
<code>subscription_contracts/update</code>	Status changes	Handle pause/cancel
<code>app/uninstalled</code>	Cleanup	Remove sessions, mark inactive

Webhook Validation

All webhooks must be HMAC-SHA256 validated using `SHOPIFY_WEBHOOK_SECRET`.

See `shopify-webhooks.server.js` for implementation pattern.

Metafield Structure

Customer Metafields (namespace: `luciteria_collection`)

Key	Type	Example	Purpose
<code>collection_type</code>	<code>single_line_text_field</code>	<code>"lucite"</code>	Active collection type
<code>owned_elements</code>	<code>json</code>	<code>["Sr","Na","Ce"]</code>	Element symbols owned
<code>wishlist_skus</code>	<code>json</code>	<code>["Sr2x2","Na2x2"]</code>	Wishlisted product SKUs
<code>preferences</code>	<code>json</code>	<code>{duplicateHandling:"never"}</code>	Assignment preferences
<code>display_name</code>	<code>single_line_text_field</code>	<code>"The Completionist"</code>	Custom display name
<code>onboarded_at</code>	<code>single_line_text_field</code>	<code>"2025-01-15T00:00:00Z"</code>	When they selected type
<code>grandfather_locked</code>	<code>single_line_text_field</code>	ISO date	Price lock date
<code>grandfather_expires</code>	<code>single_line_text_field</code>	ISO date	Price lock expiry

Product Metafields (namespace: `luciteria_collection`)

Key	Type	Example	Purpose
<code>element_symbol</code>	<code>single_line_text_field</code>	<code>"Sr"</code>	Chemical symbol
<code>atomic_number</code>	<code>number_integer</code>	<code>38</code>	Atomic number
<code>subscription_cost</code>	<code>number_decimal</code>	<code>42.00</code>	Cost basis
<code>subscription_eligible</code>	<code>boolean</code>	<code>true</code>	Can be sent via sub
<code>rarity_tier</code>	<code>single_line_text_field</code>	<code>"uncommon"</code>	Rarity classification

Data Sync Strategy

Initial Sync (App Installation)

1. Fetch all products from Shopify → populate Product table
2. Fetch all customers with subscriber tag → populate Customer table
3. Load metafields for each customer → populate collection/preference data
4. Register all webhooks

Ongoing Sync

- **Webhooks** handle real-time updates (products, inventory, orders)
- **Polling** job runs every 15 minutes as a safety net
- **Manual sync** button in admin dashboard for force-refresh

Conflict Resolution

- Shopify is the source of truth for: price, inventory, customer email
- Cabinet is the source of truth for: collection type, owned elements, preferences
- Write collection data back to Shopify via metafields

Error Handling

API Rate Limits

- Shopify REST: 40 requests per app per store per minute (bucket with leak)
- Shopify GraphQL: 1000 cost points per second
- Use exponential backoff with `withRetry()` from `error-handling.server.js`

Graceful Degradation

- If Shopify API is down, use cached local data
- Assignment engine operates on local database, not live API
- Log all API failures to `WebhookEventLog`

SKU Mapping

The real product catalog is loaded from `data/sku-mapping-master.csv`.
This file was generated from the Shopify Products export.

Collection Type Mapping

CSV Collection Type	App Type	Description
Lucite 50mm	lucite	50mm acrylic cubes with embedded samples
10mm Cube	10mm	10mm solid metal cubes
25.4mm Cube	25.4mm	1-inch solid metal cubes
50mm Cube	50mm	50mm solid metal cubes
Ampoule	ampoules	Sealed glass ampoules

SKU Naming Convention

- {Symbol}{Format} — e.g. Sr2x2 (Strontium 50mm), Fe10.1mm (Iron 10mm)
- _film suffix = film/representation (synthetic elements)
- _amp suffix = ampoule format
- _mp suffix = mirror polish variant

Key Files Reference

File	Purpose
app/config/environment.server.js	Environment config + feature flags
app/config/constants.server.js	Business rule constants
app/data/mock-db.server.js	Prototype data layer (replace in prod)
app/data/product-catalog.server.js	Real SKU catalog loader
app/data/elements.server.js	118-element periodic table data
app/lib/assignment-engine.server.js	Core assignment logic
app/lib/billing.server.js	Billing calculations
app/lib/error-handling.server.js	Error classes + logging
app/lib/data-access.server.js	Prisma queries (production)
app/integrations/shopify/*.server.js	Shopify API stubs
prisma/schema.prisma	Database schema

Testing

Prototype Mode Testing

```
APP_MODE=prototype npx remix vite:dev --port 3000
```

All routes work with mock data + real SKU catalog.

Production Mode Testing

1. Set up a Shopify development store
2. Install the app via OAuth
3. Run initial data sync
4. Verify all routes work with real API data
5. Test webhook handling with Shopify CLI